

Embedding Device Identity for ATO Detection

Is an unfamiliar device a new phone — or an attacker?

- Tested five embedding strategies across three attack types
- The right signal and training config matter more than model choice
- One failure mode is completely silent on standard metrics

Verdict: FastText mean-pool on structured feature tokens · AUC 0.985 novel · 0.798 spoof

The Key Metric: Spoof AUC

Attacker mimics OS, browser, language — differs only on timezone or network.

Attack	Difficulty	Why
Novel	Easy	Any "new device" signal works
Fleet/reuse	Medium	Requires cross-account signal
Spoof	Hard	5 of 6 features match; one mismatch carries all signal

High novel AUC is not enough. If a signal fails on spoof, it fails in production.

Embedding Device IDs Directly — Both Fail

FastText on device IDs

- Character n-grams on `dev_a3f9b2...` share substrings across accounts
- Silhouette **-0.051** — devices closer to *other* accounts than their own

Word2Vec on device IDs

- Clean clusters (silhouette +0.941), fleet AUC **0.891**
- Every new device is OOV → all get the global mean vector
- Novel attack, spoof attack, and enrollment are **identical to the model**

OOV ceiling is AUC 0.750 — structural, not tunable. Word2Vec on device IDs is offline-only.

Concatenated Feature String — Below the Trivial Baseline

Token: `ios_safari_utc-5_en_wifi_1920x1080` → one FastText embedding

- Spoof AUC **0.763** — *below* a two-line hash lookup (0.791)
- Cross-boundary n-grams (`ari_ut` , `i_utc`) bleed across feature boundaries
- Timezone mismatch gets diluted by five matching features around it

Evidence	Result
Shift timezone to different string positions	AUC 0.655–0.713 at all positions
Widen window (w=1 → w=6)	Recovers only 43.6% of mean-pool gap
Bootstrap CI vs. mean-pool	[+0.034, +0.077] — gap is structural

Mean-Pool FastText on Feature Tokens

Six tokens · embed independently · average the vectors

Attack	AUC	vs. Trivial
Novel	0.985	+0.235
Fleet	0.920	+0.170
Spoof	0.798	+0.007 ✓

- **No OOV problem** — `os_ios` is always a known token
- **Independent signal** — mismatched timezone isn't diluted by matching features
- **N-grams help** — `chrome_v120` lands near `chrome_v119` via shared substrings

Only configuration that beats the trivial baseline on all three attack types.

! The Silent Failure Mode

An earlier investigation concluded concat beats mean-pool. It was wrong — here's why.

Config	Within-feature sim	Spoof AUC
CBOW + per-event corpus	0.9993 — collapse	0.384
Skip-gram + per-account corpus	0.392	0.818

Why it collapses: Per-event corpus puts `tz_utc-5` and `tz_utc-8` in the same positional context every time. CBOW sees identical context → identical gradients → identical vectors.

Novel AUC = 0.880, fleet AUC = 0.922 even under full collapse. Only spoof drops. A standard AUC dashboard would not catch this — run a within-feature sim check after every retrain.

Four Decisions That Change the Conclusion

Decision	Wrong	Right
Training objective	CBOW	Skip-gram
Corpus construction	Per-event	Per-account
Feature representation	Concat string	Mean-pool tokens
Model on device IDs	FastText	Word2Vec

These are not tuning knobs. Each one determines whether the signal collapses silently or works at all.

Production Architecture

```
Login (real-time):
  embedding = mean(fasttext[os_X, br_X, tz_X, lang_X, net_X, sc_X])
  risk_score = cosine_distance(embedding, account_centroid)

Retraining (monthly):
  FastText: sg=1, per-account corpus, window=6, vector_size=64
  ✓ Health check: within-feature sim < 0.5 for all dims → halt if fails

Fleet detection (offline batch):
  Word2Vec on device ID sequences → review queue

Fallback:
  Per-account known-device hash set
```

Mean-pool spoof margin over trivial baseline is +0.007 — real but narrow. Measure spoof AUC explicitly in any A/B before cutover.